

Delimitação de originalidade

Fábrica de software multi-agente — a origem da proposta e sua posição no estado da arte

Enzo Consulo · 29 de agosto de 2026 · repositório público: github.com/enzoconsulo/Generic-Multi-Agents

Este documento responde a uma pergunta feita em orientação: se já existem fábricas de software agênticas, o que este trabalho acrescenta. A resposta não passa por comparar funcionalidades — passa por explicitar **de onde a proposta veio**. Ela não nasceu de um produto observado; nasceu de um diagnóstico, registrado na documentação do projeto antes de existir código. É desse diagnóstico que todos os mecanismos descem, e é por ele que a comparação com o estado da arte deve ser feita.

1. A origem da proposta: um diagnóstico, não um produto

A Parte I da documentação do trabalho abre com uma observação sobre o regime em que modelos de linguagem falham. Um modelo escreve bem quando o pedido é curto e tudo o que ele precisa saber cabe numa conversa; pedir “construa este sistema” é outra coisa, porque o trabalho dura horas, atravessa dezenas de arquivos e depende de decisões tomadas no começo que ainda precisam valer no fim. O ponto de partida é a frase que organiza o trabalho inteiro: **as falhas não são de escrita, são de processo**. O modelo continua produzindo bem cada trecho isolado; o que se perde é a coerência do conjunto.

Do diagnóstico saem quatro modos de falha, e nenhum deles se resolve com um modelo melhor:

Modo de falha	Como se manifesta
Perda de contexto	A conversa cresce, o começo sai de vista, e o modelo contradiz o que ele mesmo decidiu horas antes.
Decisão sem rastro	O motivo de uma escolha ficou no meio do diálogo. Quando a sessão termina, some junto — e a decisão é tomada de novo, às vezes ao contrário.
Autoaprovação	Quem escreveu o código é quem afirma que está pronto. O mesmo raciocínio que produziu o erro é o que vai procurá-lo.
Tentativa sem limite	Um ponto difícil consome tentativa após tentativa, trava a fila, e o custo cresce sem que nada avance.

A resposta proposta é não pedir o sistema inteiro de uma vez: dividir o pedido em tarefas pequenas e nomeadas, dar a cada uma um estado explícito registrado fora da conversa, e passar cada entrega por revisores que não a construiram. Daí saem as três regras que a documentação apresenta como origem de todo o resto — **quem implementa nunca é quem aprova; estado fora da conversa; e falha limitada**. E daí sai a tese: o gargalo de um sistema como este não está na geração de texto, e sim na governança.

A TESE DA SEGUNDA VERSÃO Se a governança é o gargalo, então governança escrita em prosa dentro de um prompt é governança opcional. A segunda versão move para o compilador, para a árvore de supervisão e para o esquema do banco tudo o que hoje é frase imperativa dirigida a um modelo. Ela não precisa ser mais esperta que a primeira: precisa ser mais difícil de operar errado.

2. Cada mecanismo é derivável de um modo de falha

Esta é a propriedade que sustenta a originalidade da proposta, e ela é verificável: não há mecanismo no sistema que não responda a um dos quatro modos. A derivação está escrita antes do código, e o commit que a implementa vem depois.

Modo de falha	O mecanismo que responde	Commit
---------------	--------------------------	--------

Perda de contexto	O agente começa frio por desenho, e recebe um índice denso do projeto em vez de varrer arquivos; cada papel recebe só o que usa.	8215dbf, 1a41ca8
Decisão sem rastro	Toda decisão é gravada com o motivo no instante em que acontece; a história do projeto vira índice consultável.	9d43b61
Autoaprovação	Dois portões operados por quem não construiu — e a ferramenta de escrever não existe para eles.	8017c41, e0bffee
Tentativa sem limite	Três ciclos por tarefa, escada de resposta ao fracasso com replanejamento, e teto de custo com parada limpa.	1a41ca8, f5b8fd4

A consequência metodológica importa mais que a lista: um mecanismo que não se ligue a um modo de falha nomeado é candidato a remoção, e um modo de falha sem mecanismo é lacuna declarada. O desenho fica auditável.

3. O estado da arte, lido pelo mesmo diagnóstico

Existem sistemas próximos, e eles precisam ser citados. Mas a pergunta útil não é “quem também tem portão de qualidade?” — praticamente todos têm. A pergunta é **qual modo de falha cada projeto resolve, e a que custo**.

3.1 Os projetos considerados

- **Agentic Software Factory (softwarefabrik.io)** — produto comercial, de Martin Janda, v0.30.0 (2026). Assistente de entrada com dezoito modelos de stack, execução em contêiner isolado, e portão final com seis papéis revisores. É o sistema mais próximo em vocabulário.
- **ChatDev e MetaGPT** (2023) — origem acadêmica da ideia de papéis especializados de agentes produzindo software, organizados em ciclo de vida e conduzidos por diálogo entre pares de agentes.
- **Cortex, Sagents, SwarmEx, Synapse** — bibliotecas de orquestração de agentes em Elixir/OTP, nas quais agente como processo supervisionado já é prática corrente.
- **Factory.ai e correlatos** — posicionamento comercial de fábrica agêntica; e, antes de tudo isso, o próprio termo “fábrica de software” vem da engenharia de software das décadas de 1960 a 1990 (CUSUMANO, 1991).

3.2 Quem resolve o quê

Modo de falha	Este trabalho	softwarefabrik.io	ChatDev / MetaGPT
Perda de contexto	Agente frio por desenho; índice denso; contexto por papel	O operador humano sustenta a continuidade entre passos	O histórico de diálogo é o estado — o modo não é atacado
Decisão sem rastro	Decisão gravada com o motivo; história indexada	Versionamento e artefatos do espaço de trabalho	Não trata
Autoaprovação	Portões sem a ferramenta de escrever	Resolvido pela aprovação humana, ao custo da autonomia	Revisor é agente do mesmo diálogo
Tentativa sem limite	Três ciclos, escada de fracasso, teto por tarefa	O operador decide quando parar	Novo ciclo de diálogo

A leitura da tabela é o argumento central deste documento. O sistema comercial mais próximo resolve a autoaprovação **colocando um humano no portão** — solução legítima, e a mais segura, mas que responde a uma pergunta diferente da que este trabalho faz. Os sistemas acadêmicos, ao manterem o diálogo como estado, atacam a autoaprovação e deixam a perda de contexto praticamente intocada, que é o primeiro dos quatro modos.

4. As diferenças reais

4.1 A autoaprovação é resolvida por ausência de ferramenta, não por presença de humano

Esta é a diferença que decide, porque as duas soluções são mutuamente exclusivas. Colocar o humano no portão resolve a autoaprovação e **elimina a autonomia**; retirar a ferramenta de escrever do revisor resolve a autoaprovação **preservando a autonomia**, e transfere o problema para outro lugar — é preciso então limitar a falha por desenho,

porque não há quem interrompa. Todo o limite de ciclos, a escada de resposta ao fracasso e o replanejamento automático existem por causa dessa escolha.

A regra deixa de ser interpretável e passa a ser impossibilidade: o catálogo de ferramentas do revisor não contém escrita, e há teste que percorre os papéis para garantir isso. É a tese do trabalho — governança como propriedade da estrutura, não como frase — aplicada ao próprio sistema.

4.2 O coordenador não é um agente

Nos sistemas acadêmicos a coordenação acontece por diálogo entre agentes; no sistema comercial, quem coordena é o operador. Aqui, quem decide ordem, promoção, roteamento e escalonamento é código determinístico. A decisão foi tomada **por medição, não por preferência**: observou-se que o coordenador baseado em modelo consumia cerca de 17% do custo de um trabalho para executar o que é, literalmente, uma máquina de estados. A fronteira adotada é explícita — cabe num teste, então é código; replanejar e julgar um marco reprovado continuam no modelo, porque são julgamento.

4.3 Não há catálogo nem elenco: o sistema sintetiza o que precisa

De uma frase em linguagem natural saem especificação, plano em fases e de oito a vinte tarefas com dependências declaradas; a equipe é sintetizada em dois a cinco especialistas a partir do próprio pedido. O sistema comercial parte de dezoito modelos de stack; os acadêmicos, de um elenco fixo de papéis de empresa. A diferença não é de conveniência: um catálogo delimita o que o sistema aceita construir, enquanto a síntese desloca esse limite para a capacidade de decompor.

4.4 O desenho é derivado da telemetria da própria operação

Esta é a contribuição empírica, e é a que nenhum dos projetos comparados pode ter, porque depende de dados que só existem neste repositório. A primeira versão operou seis semanas em uso real, com contabilidade por chamada, e a segunda é reprojetada a partir do que foi medido — inclusive quando a medição contrariou a hipótese de partida. Os sistemas acadêmicos medem qualidade de saída em conjuntos de referência; o comercial estima custo antes de executar. **Nenhum publica medição da própria operação para reprojetar-se.**

SOBRE A EXTENSÃO A ARTEFATOS NÃO-SOFTWARE O sistema também opera fora de software, com uma escada de prova que obriga o verificador a declarar se o critério foi executado, inspecionado ou julgado. É extensão do mesmo princípio — se a prova não vem de graça, ela tem de ser declarada — e vale registro, mas não é a contribuição central deste trabalho.

5. Convergência independente, e não plágio

Há sobreposição real com o estado da arte, e ela deve ser declarada sem eufemismo: o termo “fábrica de software” é anterior à era dos modelos de linguagem; papéis especializados de agentes vêm de ChatDev e MetaGPT; portão de qualidade, um commit por unidade de trabalho, modelo diferente por papel e estimativa de custo existem no produto comercial; e **agente como processo supervisionado é prática corrente na comunidade Elixir**, não invenção deste trabalho. São itens a citar, não a reivindicar.

Três argumentos sustentam que a semelhança é convergência, e não cópia:

- **O mesmo diagnóstico, sob as mesmas restrições, produz mecanismos parecidos.** Quem trata quatro modos de falha conhecidos, sobre a mesma família de modelos e com controle de versão como substrato, chega a portões, a limites de tentativa e a separação de papéis. Convergência aqui é **corroboração do diagnóstico**, e é assim que a literatura costuma tratar resultados independentes que se encontram.

- **A cronologia mostra derivação, não transcrição.** Os conceitos aparecem no repositório em sequência, cada um depois do incidente que o motivou, ao longo de seis semanas. Um desenho transcrito chega inteiro; este chegou por acúmulo, e a seção 6 mostra o registro.
- **As soluções divergem onde o problema é o mesmo.** Autoaprovação é atacada por todos; este trabalho é o único que a resolve sem colocar um humano no caminho, e paga por isso com maquinaria própria de limitação de falha. Divergência sob problema comum é o oposto de cópia.

POSICIONAMENTO SUGERIDO O trabalho pode e deve ser apresentado em diálogo com a literatura, e não em oposição a ela: adota o diagnóstico e o vocabulário já estabelecidos, herda mecanismos citados nominalmente, e investiga uma pergunta que os antecessores não fazem — se é possível manter a separação entre construir e aprovar sem operador humano no laço, e o que isso custa. É contribuição incremental declarada, que é o que se espera de um trabalho de conclusão.

6. Evidência documental

O repositório é público e a verificação é direta. Entre 18 de julho e 28 de agosto de 2026 acumulou 205 commits. Os mecanismos centrais têm commit e data:

Commit	Data	O que registra
8215dbf	01/08	Índice denso do projeto no lugar da varredura de arquivos
9d43b61	01/08	Doutrina de custo de contexto: o modelo, a medição e as intervenções
f5b8fd4	01/08	Limiar medido de tamanho de tarefa, antes do despacho
1a41ca8	02/08	Montador de contexto por papel e teto de custo com parada limpa
91559b8	02/08	O laço de orquestração vira código testado
8017c41	02/08	O pipeline decide sozinho, sem portão de julgamento humano
cdedec2	02/08	Saneamento passa a consultar o histórico de versões, não as notas
e0bffee	23/08	Fechamento do portão nos dois caminhos de saída do construtor

Medições que contrariaram a própria hipótese

O sinal de que o método é honesto não é a medição que confirma o que já se supunha, e sim a que obriga a mudar de rumo. Quatro casos, todos registrados em commit:

- **174f0a0 (31/07) — previsões escritas antes da rodada**, para poder confrontá-las com o resultado depois.
- **696762e (31/07) — um ganho de 12% foi remedido e identificado como ruído**, e a afirmação foi retirada.
- **dea9e9c (31/07) — a contagem de saída estava cem vezes subestimada**. O instrumento estava errado, e foi corrigido antes de qualquer conclusão.
- **e8974c5 (21/08) — hipótese de gargalo avaliada e descartada**: supunha-se que o portão de verificação limitaria a vazão; medido sobre quarenta e três rodadas, houve zero segundo de espera, e a otimização já desenhada foi abandonada.

Operação real

A primeira versão entregou 86 tarefas concluídas de 89, em três projetos de domínios distintos — um jogo de tabuleiro em versão web multijogador em tempo real; um serviço permanente embarcado no computador de placa única que controla uma impressora 3D; e uma plataforma de dados com camada de análise por modelos de linguagem. São 139 execuções gravadas, com tokens, custo, modelo e desfecho, preservadas no repositório.

7. Encaminhamento

- **Escrever a seção de trabalhos relacionados e citar antes de ser questionado:** ChatDev e MetaGPT pelos papéis; softwarefabrik.io e Factory.ai pela fábrica agêntica; Cortex, Sagents, SwarmEx e Synapse pelo agente como processo supervisionado; Cusumano pelo termo.
- **Enunciar a contribuição pela pergunta, não pelo artefato:** não “construí uma fábrica de software multi-agente”, e sim “investiguei se a separação entre construir e aprovar se sustenta sem operador humano no laço, e medi o que isso custa”.
- **Apresentar a derivação da seção 2 como método.** Um desenho em que cada mecanismo se liga a um modo de falha nomeado é defensável de forma que uma lista de funcionalidades não é.

Referências

- CUSUMANO, M. Japan's Software Factories. Oxford University Press, 1991.
- QIAN, C. et al. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924, 2023.
- HONG, S. et al. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework, 2023.
- JANDA, M. Agentic Software Factory (softwarefabrik.io), v0.30.0, 2026.
- Factory.ai. Factory 2.0: from coding agents to software factories.
- Cortex (github.com/itsHabib/cortex) e Sagents (github.com/sagents-ai/sagents) — orquestração de agentes em Elixir/OTP.